

MODUL 8

Nama: Ryan Akeyla Novianto Widodo

Kelas: 12 IF 07

NIM : 103112400081

DASAR TEORI

Dasar Teori C++

Diambil dari internet dan dirangkum ulang oleh saya dan dibantu dengan AI.

1. Inti Program

- Syntax: Mirip bahasa Inggris, tapi huruf besar dan kecil beda arti. Setiap perintah biasanya diakhiri dengan titik koma (;).

- Komentar: Catatan di kode yang tidak dijalankan.

- Satu baris: // Ini komentar

- Banyak baris: /* Ini komentar yang panjang */

- Header: File yang berisi daftar perintah dan fungsi yang bisa dipakai. Contoh:

cpp

```
#include <iostream> // Untuk perintah input/output (cout, cin)
```

- main() : Fungsi utama, tempat program mulai berjalan.

cpp

```
int main() {
```

```
    // Kode program di sini
```

```
    return 0; // Menandakan program selesai dengan benar
```

```
}
```

- Namespace: Seperti "area" untuk nama-nama perintah. std adalah namespace standar C++.

cpp

```
using namespace std; // Supaya tidak perlu tulis std::cout, std::cin, dll.
```

- Input/Output:

- cout : Menampilkan sesuatu di layar.

- cin : Membaca input dari keyboard.

cpp

```
cout << "Halo!" << endl; // Menampilkan "Halo!"
```

```
int umur;
```

```
cin >> umur; // Membaca angka dari keyboard dan disimpan di variabel "umur"
```

2. Jenis Data

- Integer (Bilangan Bulat):

- int : Bilangan bulat (contoh: -10, 0, 25).

- short : Bilangan bulat pendek.

- long : Bilangan bulat panjang.

- Floating-Point (Bilangan Desimal):

- float : Bilangan desimal (contoh: 3.14).

- double : Bilangan desimal dengan ketelitian lebih tinggi.

- Character (Karakter):

- char : Satu huruf, angka, atau simbol (contoh: 'a', 'Z', '5').

- Boolean (Logika):

- bool : Nilai benar (true) atau salah (false).

- void : "Kosong". Dipakai untuk fungsi yang tidak menghasilkan nilai.

3. Variabel

- Deklarasi: Memberi tahu program nama dan jenis data variabel.

cpp

```
int umur; // Variabel bernama "umur" untuk menyimpan bilangan bulat
```

```
double harga; // Variabel bernama "harga" untuk menyimpan bilangan desimal
```

- Inisialisasi: Memberi nilai awal ke variabel.

cpp

```
int umur = 20; // "umur" diisi dengan nilai 20
```

```
double harga = 99.50; // "harga" diisi dengan nilai 99.50
```

4. Operator

- Aritmatika: + (tambah), - (kurang), * (kali), / (bagi), % (sisanya).
- Penugasan: = (mengisi nilai ke variabel).
- Perbandingan: == (sama dengan), != (tidak sama dengan), > (lebih besar), < (lebih kecil), >= (lebih besar atau sama dengan), <= (lebih kecil atau sama dengan).
- Logika: && (DAN), || (ATAU), ! (TIDAK).
- Increment/Decrement: ++ (tambah 1), -- (kurang 1).

5. Struktur Kontrol

- if : Melakukan sesuatu jika kondisi benar.

cpp

```
if (umur >= 17) {  
    cout << "Anda sudah dewasa." << endl;  
} else {  
    cout << "Anda belum dewasa." << endl;  
}
```

- switch : Memilih salah satu dari beberapa pilihan.

cpp

```
int hari = 3;  
switch (hari) {  
    case 1:  
        cout << "Senin" << endl;  
        break;  
    case 2:
```

```

        cout << "Selasa" << endl;

        break;

    case 3:

        cout << "Rabu" << endl;

        break;

    default:

        cout << "Hari lain" << endl;

}

```

- for : Melakukan sesuatu berulang kali sebanyak yang ditentukan.

cpp

```

for (int i = 0; i < 5; i++) {

    cout << i << " "; // Menampilkan angka 0 sampai 4

}

cout << endl;

```

- while : Melakukan sesuatu berulang kali selama kondisi benar.

cpp

```

int angka = 1;

while (angka <= 3) {

    cout << angka << " ";

    angka++;

}

cout << endl;

```

- do-while : Sama seperti while , tapi dilakukan minimal sekali.

cpp

```

int pilihan;

do {

    cout << "Pilih 1, 2, atau 3: ";

```

```
cin >> pilihan;  
} while (pilihan < 1 || pilihan > 3);
```

6. Fungsi

- Definisi: Sekumpulan kode yang melakukan tugas tertentu.

cpp

```
int tambah(int a, int b) {  
    return a + b;  
}
```

- Pemanggilan: Menggunakan fungsi.

cpp

```
int hasil = tambah(5, 2); // Memanggil fungsi "tambah" dengan angka 5 dan 2  
cout << "Hasil: " << hasil << endl; // Menampilkan "Hasil: 7"
```

7. Pointer

- Definisi: Variabel yang menyimpan alamat memori variabel lain. (Agak rumit, tapi penting).

cpp

```
int umur = 25;  
  
int *p_umur = &umur; // p_umur menyimpan alamat memori dari variabel umur
```

DASAR TEORI QUEUE

Pengertian Queue

Queue adalah struktur data linear yang bekerja dengan prinsip FIFO (First In, First Out), yaitu elemen yang pertama kali masuk akan pertama kali keluar. Dalam bahasa pemrograman C++, queue digunakan untuk mengelola data yang diproses secara berurutan, seperti pada sistem antrian.

Queue diimplementasikan menggunakan struktur seperti array, linked list, atau library STL `<queue>`. Operasi utama pada queue meliputi enqueue (menambahkan data ke antrian), dequeue (menghapus data dari antrian), peek/front (melihat data terdepan), serta pengecekan apakah queue kosong atau penuh.

Queue banyak digunakan dalam berbagai aplikasi, seperti manajemen antrian, penjadwalan proses, dan algoritma Breadth First Search (BFS).

Singkatnya sih Antrian

Guided 1

```
#include <iostream>
using namespace std;

#define MAX 5 // ukuran maksimal queue

// Struktur Queue
struct Queue {
    int data[MAX];
    int head;
    int tail;
};

//Membuat antrean kosong
void createQueue(Queue &Q) {
    Q.head = -1;
    Q.tail = -1;
}

// Mengecek apakah queue kosong
bool isEmpty(Queue Q) {
    return (Q.head == -1 && Q.tail == -1);
}

// Mengecek apakah queue penuh
bool isFull(Queue Q) {
    return (Q.tail == MAX -1);
}

// Menampilkan isi antrean
void printQueue(Queue Q) {
    if (isEmpty(Q)) {
        cout << "Queue kosong!" << endl;
    } else {
        cout << "Queue : ";
        for (int i= Q.head; i <= Q.tail; i++) {
            cout << Q.data[i] << " ";
        }
        cout << endl;
    }
}

// Menambahkan elemen ke dalam antrean (Enqueue)
void enqueue(Queue &Q, int x) {
    if (isFull(Q)) {
        cout << "Queue penuh! Tidak bisa menambah data." << endl;
    } else {
        if (isEmpty(Q)) {
            Q.head = Q.tail = 0;
        }
    }
}
```

```

    } else {
        Q.tail++;
    }
    Q.data[Q.tail] = x;
    cout << "Enqueue: " << x << endl;
}
}

// Menghapus elemen dari elemen antrean (Dequeue)
void dequeue(Queue &Q) {
    if (isEmpty(Q)) {
        cout << "Queue kosong! Tidak ada yang dihapus." << endl;
    } else {
        cout << "Dequeue: " << Q.data[Q.head] << endl;
        //Jika hanya 1 elemen
        if (Q.head == Q.tail) {
            Q.head = Q.tail = -1;
        } else {
            //Geser semua elemen ke kiri
            for (int i = Q.head; i < Q.tail; i++) {
                Q.data[i] = Q.data[i+1];
            }
            Q.tail--;
        }
    }
}

int main() {
    Queue Q;
    createQueue(Q);

    enqueue(Q, 5);
    enqueue(Q, 2);
    enqueue(Q, 7);
    printQueue(Q);

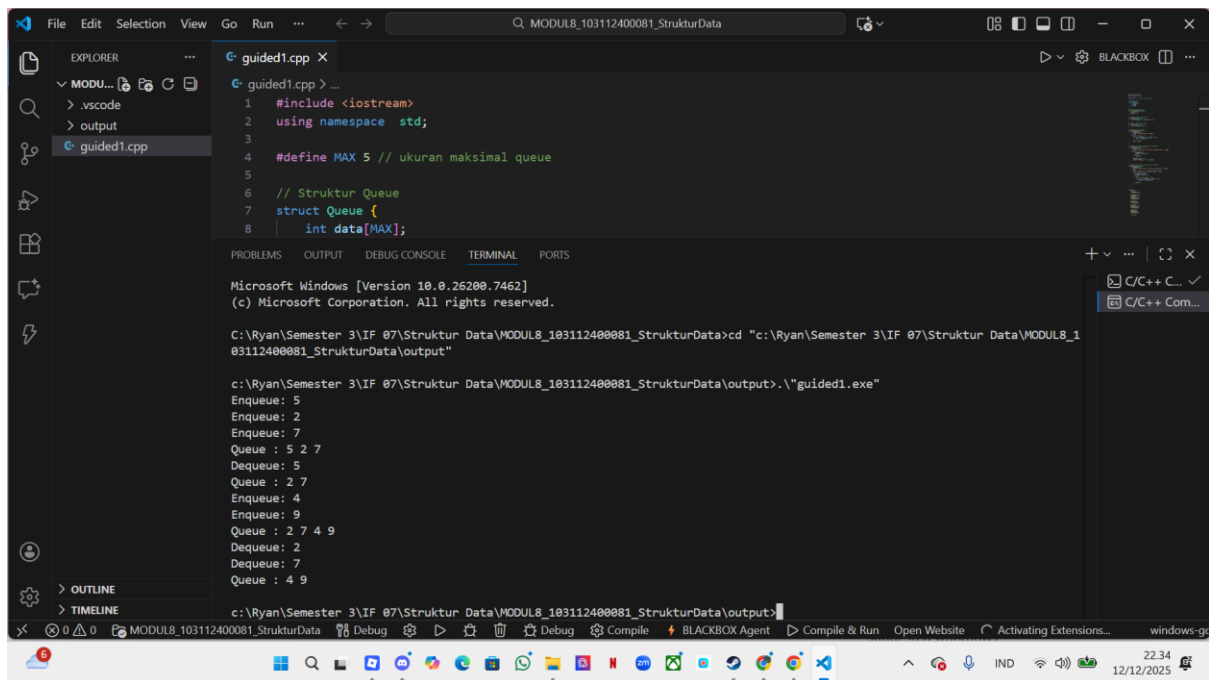
    dequeue(Q);
    printQueue(Q);

    enqueue(Q, 4);
    enqueue(Q, 9);
    printQueue(Q);

    dequeue(Q);
    dequeue(Q);
    printQueue(Q);

    return 0;
}

```

```
guided1.cpp X
C: \Ryan\Semester 3\IF 07\Struktur Data\MODUL8_103112400081_StrukturData>cd "c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL8_103112400081_StrukturData\output"
c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL8_103112400081_StrukturData\output>.\"guided1.exe"
Enqueue: 5
Enqueue: 2
Enqueue: 7
Queue : 5 2 7
Dequeue: 5
Queue : 2 7
Enqueue: 4
Enqueue: 9
Queue : 2 7 4 9
Dequeue: 2
Dequeue: 7
Queue : 4 9
```

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita mengimplementasikan struktur data Queue (antrian) menggunakan array dalam bahasa C++, serta melakukan operasi dasar pada queue seperti:

1. Membuat antrean kosong

Menginisialisasi head dan tail dengan -1 sebagai tanda queue masih kosong.

2. Melakukan operasi enqueue

Menambahkan elemen ke bagian belakang antrian (tail bergerak maju).

3. Melakukan operasi dequeue

Menghapus elemen bagian depan antrian (head), lalu menggeser elemen lain ke kiri.

4. Mengecek kondisi queue

- isEmpty() → mengecek apakah queue kosong
- isFull() → mengecek apakah queue penuh

5. Menampilkan isi queue

Menunjukkan seluruh elemen antrian dari head sampai tail.

Unguided

Coding:

```
#include <iostream>
using namespace std;

const int MAX = 5;

struct Queue {
    int info[MAX];
    int head;
    int tail;
};

void createQueue1(Queue &Q) {
    Q.head = 0;
    Q.tail = -1;
}

bool isEmptyQueue1(Queue Q) {
    return Q.tail < Q.head;
}

bool isFullQueue1(Queue Q) {
    return Q.tail == MAX - 1;
}

void enqueue1(Queue &Q, int x) {
    if (isFullQueue1(Q)) {
        cout << "Queue penuh!" << endl;
        return;
    }
    Q.tail++;
    Q.info[Q.tail] = x;
}

int dequeue1(Queue &Q) {
    if (isEmptyQueue1(Q)) {
        cout << "Queue kosong!" << endl;
        return -1;
    }
    int x = Q.info[Q.head];
    Q.head++;
    return x;
}

void printInfo1(Queue Q) {
    cout << "H=" << Q.head << " T=" << Q.tail << " | ";
    if (isEmptyQueue1(Q)) {
        cout << "Queue kosong." << endl;
    }
}
```

```

        return;
    }
    for (int i = Q.head; i <= Q.tail; i++) {
        cout << Q.info[i] << " ";
    }
    cout << endl;
}

void createQueue2(Queue &Q) {
    Q.head = -1;
    Q.tail = -1;
}

bool isEmptyQueue2(Queue Q) {
    return Q.head == -1;
}

bool isFullQueue2(Queue Q) {
    return Q.tail == MAX - 1;
}

void enqueue2(Queue &Q, int x) {
    if (isFullQueue2(Q)) {
        cout << "Queue penuh!" << endl;
        return;
    }
    if (isEmptyQueue2(Q)) {
        Q.head = 0;
        Q.tail = 0;
    } else {
        Q.tail++;
    }
    Q.info[Q.tail] = x;
}

int dequeue2(Queue &Q) {
    if (isEmptyQueue2(Q)) {
        cout << "Queue kosong!" << endl;
        return -1;
    }
    int x = Q.info[Q.head];

    for (int i = Q.head; i < Q.tail; i++) {
        Q.info[i] = Q.info[i + 1];
    }
    Q.tail--;

    if (Q.tail < 0) {
        Q.head = -1;
        Q.tail = -1;
    }
}

```

```

    }

    return x;
}

void printInfo2(Queue Q) {
    cout << "H=" << Q.head << " T=" << Q.tail << " | ";
    if (isEmptyQueue2(Q)) {
        cout << "Queue kosong." << endl;
        return;
    }
    for (int i = Q.head; i <= Q.tail; i++) {
        cout << Q.info[i] << " ";
    }
    cout << endl;
}

void createQueue3(Queue &Q) {
    Q.head = -1;
    Q.tail = -1;
}

bool isEmptyQueue3(Queue Q) {
    return Q.head == -1;
}

bool isFullQueue3(Queue Q) {
    return (Q.tail + 1) % MAX == Q.head;
}

void enqueue3(Queue &Q, int x) {
    if (isFullQueue3(Q)) {
        cout << "Queue penuh!" << endl;
        return;
    }

    if (isEmptyQueue3(Q)) {
        Q.head = 0;
        Q.tail = 0;
    } else {
        Q.tail = (Q.tail + 1) % MAX;
    }

    Q.info[Q.tail] = x;
}

int dequeue3(Queue &Q) {
    if (isEmptyQueue3(Q)) {
        cout << "Queue kosong!" << endl;
        return -1;
    }

```

```

    }

    int x = Q.info[Q.head];

    if (Q.head == Q.tail) {
        Q.head = -1;
        Q.tail = -1;
    } else {
        Q.head = (Q.head + 1) % MAX;
    }

    return x;
}

void printInfo3(Queue Q) {
    cout << "H=" << Q.head << " T=" << Q.tail << " | ";

    if (isEmptyQueue3(Q)) {
        cout << "Queue kosong." << endl;
        return;
    }

    int i = Q.head;
    while (true) {
        cout << Q.info[i] << " ";
        if (i == Q.tail) break;
        i = (i + 1) % MAX;
    }
    cout << endl;
}

int main() {

    cout << "\n===== QUEUE ALTERNATIF 1 (HEAD DIAM, TAIL BERGERAK)
===== " << endl;
    Queue Q1;
    createQueue1(Q1);

    printInfo1(Q1);
    enqueue1(Q1, 5); printInfo1(Q1);
    enqueue1(Q1, 2); printInfo1(Q1);
    enqueue1(Q1, 7); printInfo1(Q1);
    dequeue1(Q1); printInfo1(Q1);
    enqueue1(Q1, 4); printInfo1(Q1);
    dequeue1(Q1); printInfo1(Q1);
    dequeue1(Q1); printInfo1(Q1);

    cout << "\n===== QUEUE ALTERNATIF 2 (HEAD & TAIL BERGERAK /
SHIFTING) ===== " << endl;
    Queue Q2;

```

```

createQueue2(Q2);

enqueue2(Q2, 5); printInfo2(Q2);
enqueue2(Q2, 2); printInfo2(Q2);
enqueue2(Q2, 7); printInfo2(Q2);
dequeue2(Q2); printInfo2(Q2);
enqueue2(Q2, 4); printInfo2(Q2);
dequeue2(Q2); printInfo2(Q2);
dequeue2(Q2); printInfo2(Q2);

cout << "\n===== QUEUE ALTERNATIF 3 (CIRCULAR QUEUE) =====" <<
endl;
Queue Q3;
createQueue3(Q3);

enqueue3(Q3, 5); printInfo3(Q3);
enqueue3(Q3, 2); printInfo3(Q3);
enqueue3(Q3, 7); printInfo3(Q3);
dequeue3(Q3); printInfo3(Q3);
enqueue3(Q3, 4); printInfo3(Q3);
dequeue3(Q3); printInfo3(Q3);
dequeue3(Q3); printInfo3(Q3);

return 0;
}

```

Hasil Coding

```

c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL8_103112400081_StrukturData\output>.\"Unguided.exe"

===== QUEUE ALTERNATIF 1 (HEAD DIAM, TAIL BERGERAK) =====
H=0 T=-1 | Queue kosong.
H=0 T=0 | 5
H=0 T=1 | 5 2
H=0 T=2 | 5 2 7
H=1 T=2 | 2 7
H=1 T=3 | 2 7 4
H=2 T=3 | 7 4
H=3 T=3 | 4

===== QUEUE ALTERNATIF 2 (HEAD & TAIL BERGERAK / SHIFTING) =====
H=0 T=0 | 5
H=0 T=1 | 5 2
H=0 T=2 | 5 2 7
H=0 T=1 | 2 7
H=0 T=2 | 2 7 4
H=0 T=1 | 7 4
H=0 T=0 | 4

===== QUEUE ALTERNATIF 3 (CIRCULAR QUEUE) =====
H=0 T=0 | 5
H=0 T=1 | 5 2
H=0 T=2 | 5 2 7
H=1 T=2 | 2 7
H=1 T=3 | 2 7 4
H=2 T=3 | 7 4
H=3 T=3 | 4

c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL8_103112400081_StrukturData\output>

```

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita mengimplementasikan struktur data Queue menggunakan array dalam bahasa C++ dengan tiga mekanisme berbeda.

1 Alternatif 1 – Head diam, Tail bergerak

Queue dibuat dengan cara:

- head tetap berada di indeks awal
- tail bergerak maju setiap elemen masuk
- elemen yang keluar cukup dengan menaikkan nilai head

Ini menunjukkan bentuk **queue linear sederhana**.

2 Alternatif 2 – Head bergerak, Tail bergerak (Shifting)

Pada metode ini:

- ketika elemen keluar (dequeue), data digeser (shift) ke kiri
- head tetap di 0 setelah digeser
- tail turun 1

Mekanisme ini memperlihatkan **queue dengan pergeseran (shifting)** untuk menghemat tempat.

3 Alternatif 3 – Head dan Tail berputar (Circular Queue)

Metode ini menggunakan operasi modulo (%) sehingga:

- head dan tail dapat kembali ke indeks 0 setelah mencapai batas array
- memaksimalkan penggunaan ruang

Ini disebut **Circular Queue**, yaitu bentuk antrian efisien yang memutar indeks array.